Deep Architectural Analysis of PostgreSQL 18 Shared Memory Infrastructure: Beyond shared_buffers

The PostgreSQL architecture relies upon a heavily engineered multi-process model, which inherently lacks a unified address space. To facilitate high-performance inter-process communication (IPC), transient state synchronization, and absolute data consistency across concurrent backend processes, the system maps a massive, centralized shared memory segment. While the majority of database administration literature and performance tuning focuses almost exclusively on shared_buffers—the primary page cache for relation data—the reality of the PostgreSQL shared memory ecosystem is vastly more complex.
In the REL_18_STABLE branch of the source code, the shared memory landscape accommodates profound architectural shifts, most notably the introduction of a native Asynchronous I/O (AIO) subsystem, the refactoring of the cumulative statistics system into Dynamic Shared Areas (DSA), and the removal of hardcoded limits on fast-path lock tracking. This technical report provides an exhaustive, source-code-level examination of the shared memory objects created during the server bootstrap sequence, excluding shared_buffers, detailing the precise engineering purpose, allocation mechanisms, and concurrency paradigms of these critical internal structures.

The Bootstrap Lifecycle and Shared Memory Allocation

The orchestration of shared memory begins strictly within the Postmaster process during the server's initialization sequence[1]. Because the Postmaster shares memory with all subsequently forked backend processes, it must adhere to a strict design contract: it must avoid touching shared memory structures during routine operation. This isolation prevents system-wide stalls or complete cluster failure if a backend process crashes or terminates ungracefully while holding a lock or corrupting a shared structure[1].
The primary entry point for shared memory allocation is the CreateSharedMemoryAndSemaphores() function, located within the src/backend/storage/ipc/ipci.c source file[2]. This routine initiates a highly deterministic, multi-phase bootstrap process to calculate, allocate, format, and populate the IPC segment before any client connections are permitted[2].

Phase 1: Deterministic Memory Estimation via CalculateShmemSize

Before any memory mapping syscalls are invoked against the operating system kernel, PostgreSQL must determine the exact, immutable byte footprint required. Because the operating system typically requires the full size of a System V (SysV) or mmap segment upon creation, dynamic expansion of the main shared memory block is natively impossible[6]. To

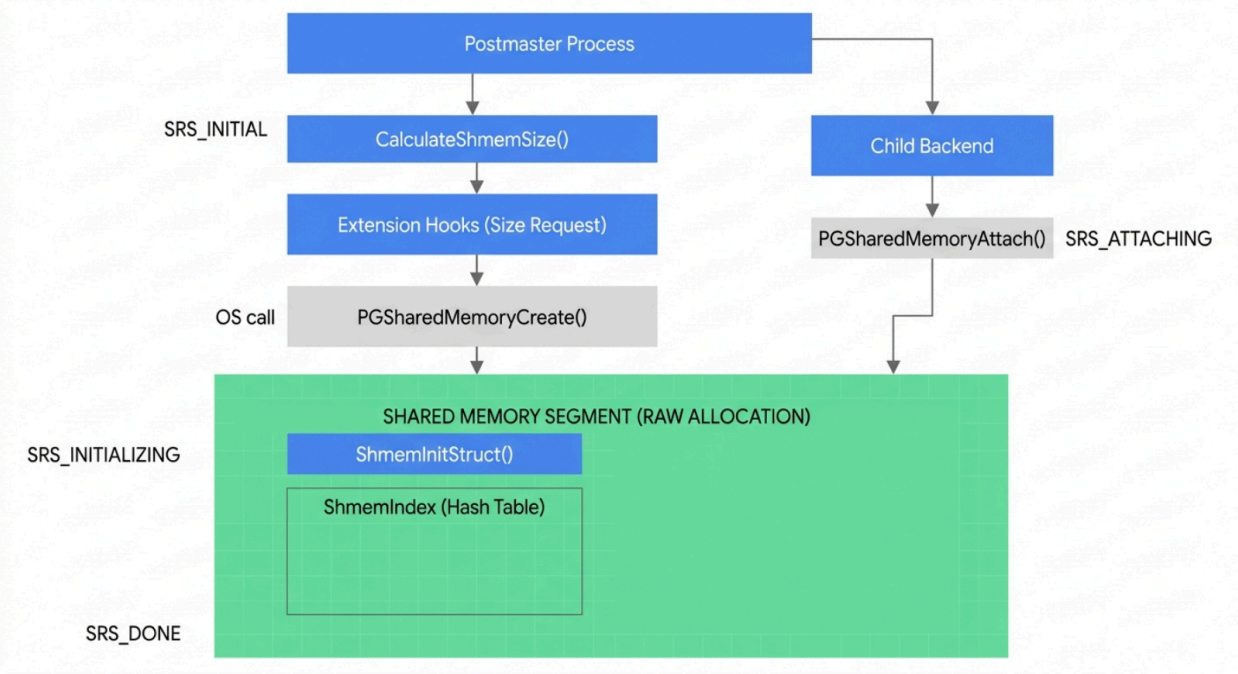
resolve this, the system invokes the CalculateShmemSize() function, which acts as a centralized aggregator of the memory requirements for every internal module[4].
The estimation phase sequentially invokes specific size-calculation routines from various subsystems, such as BufferShmemSize(), LockShmemSize(), and CalculateFastPathLockShmemSize()[8]. The logic utilizes an internal add_size(Size s1, Size s2) macro to safely accumulate the byte count while explicitly checking for size_t integer overflows, a critical security and stability measure[10]. Alongside these precise module estimates, the algorithm injects a supplementary 100,000-byte allocation designed to accommodate smaller, miscellaneous data structures that are too minor to track individually[8]. Crucially, external extensions loaded via shared_preload_libraries (such as pg_stat_statements or spatial index extensions) also inject their memory requirements during this phase through the shmem_request_hook[10]. The aggregated value of these third-party requests is tracked in the total_addin_request variable[10]. The PostgreSQL 18 architecture enforces strict initialization discipline; any code attempting to request shared memory outside of this specific hook phase will cause the server startup to abort immediately with a FATAL error[11]. Finally, the resulting total size is mathematically aligned by rounding it up to the nearest multiple of the typical system page size (typically 8,192 bytes, represented by the BLCKSZ constant)[8].

Phase 2: Segment Instantiation and the OS Contract

Once the total required size is finalized, the Postmaster creates the physical segment via the PGSharedMemoryCreate() function[6]. Depending on the operating system capabilities and the shared_memory_type configuration parameter (which can be configured to favor mmap or SysV), the memory is mapped into the process space[6].
During this instantiation, the system negotiates large memory pages (huge pages) with the OS kernel. If the huge_pages parameter is set to on, the system attempts to utilize the MAP_HUGETLB flag[6]. The use of huge pages dramatically reduces the size of the CPU's Translation Lookaside Buffer (TLB), preventing TLB cache misses when backends rapidly scan across massive memory segments[6]. If huge pages are demanded but unsupported by the current kernel or hardware platform, the startup fails with a specific ERRCODE_FEATURE_NOT_SUPPORTED exception[6].
The identity of the shared memory segment is strictly tied to the database cluster's data directory. The system utilizes the data directory's inode and device numbers (statbuf.st_ino) to generate a unique IpcMemoryKey[6]. This prevents accidental attachment by overlapping PostgreSQL installations on the same server hardware. In the event of an unclean shutdown, the Postmaster attempts to detect orphaned shared memory segments and potentially removes and recreates them to ensure a pristine initialization state[6].
Furthermore, to maintain process lifecycle hygiene, the system registers cleanup routines such as AnonymousShmemDetach and utilizes on_shmem_exit tracking arrays[6]. These arrays ensure that upon normal termination, or an induced abort via SIGQUIT, the memory mappings and OS semaphores are unlinked and destroyed, preventing memory leaks at the kernel level[6]. The creation of these semaphores is heavily reliant on kernel parameters; if PostgreSQL's requested

semaphore consumption exceeds the kernel.sem system limits (SEMMNI for maximum semaphore sets, or SEMMNS for system-wide maximum semaphores), the engine will fail to start[14]. Database administrators must frequently tune the max_connections parameter or modify the sysctl kernel configurations to accommodate the massive semaphore arrays PostgreSQL deploys to suspend waiting backend processes[14].

PostgreSQL 18 Shared Memory Bootstrap and Allocation Sequence



The lifecycle of shared memory initialization, driven by CreateSharedMemoryAndSemaphores(), progressing from hook registration and size estimation to raw allocation and named object instantiation.

Phase 3: The Internal Allocator and the ShmemIndex

With the raw byte array mapped from the operating system, PostgreSQL must manage this memory internally. Standard standard library functions like malloc() cannot be used, as they manage memory in the private address space of a single process. Instead, PostgreSQL deploys a specialized bump allocator during startup, represented by the InitShmemAllocator() and ShmemAllocRaw() functions, which establish a PGShmemHeader at the base of the shared segment[10].

A paramount challenge in IPC programming is locating specific data structures across different processes. While PostgreSQL attempts to map the shared segment at a fixed virtual memory address across all backends, Address Space Layout Randomization (ASLR) and specific operating system security constraints can force the segment to be mapped at different

absolute addresses in different processes. To solve this, PostgreSQL utilizes a centralized, string-to-pointer hash table known as the ShmemIndex[16].

When an internal subsystem or an external extension initializes its specific objects, it calls the ShmemInitStruct(const char *name, Size size, bool *foundPtr) API[16]. This function locks the ShmemIndexLock to prevent race conditions, performs a hash search (hash_search()) using the provided string name, and checks if the structure already exists[16]. If it does not, the function allocates the requested bytes from the bump allocator, casts the offset to a pointer, and stores it in the hash table alongside its identifying string[16].

This string-based indexing ensures that when the Postmaster subsequently forks child worker processes, those children can simply look up terms like "Lock Manager", "Proc Array", or "PgStat Data" in the ShmemIndex to locate the exact shared pointers. The system progresses through rigorously defined enumeration states to govern this setup, ensuring absolute memory safety before client connections are permitted.

Internal Initialization State	**Phase Description and Component Behavior**
SRS_INITIAL	The foundational state where the system has not yet begun assembling shared memory request callbacks[16].
SRS_REQUESTING	The phase where shmem_request callbacks are triggered. All structures invoke size calculation functions here[16].
SRS_INITIALIZING	The Postmaster has finalized size requests and physically mapped the memory segment. init_fn callbacks format the raw memory blocks[16].
SRS_ATTACHING	Postmaster child processes (worker backends) are starting up. attach_fn callbacks are invoked to resolve local pointers via the ShmemIndex[16].
SRS_DONE	All shared memory structures are fully initialized, mapped, and accessible. The server is ready to accept client TCP connections[16].

Concurrency Control: The Synchronization Primitives

Excluding the page cache buffers, the most critical structures residing in shared memory are those that dictate concurrency control. The PostgreSQL lock manager handles heavy transactional isolation, while lightweight locks handle immediate memory protections and CPU-level synchronization.

Lightweight Locks (LWLocks)

Lightweight Locks (LWLocks) provide high-performance, ultra-short-duration mutual exclusion for specific shared memory data structures. Unlike heavyweight transactional locks, LWLocks are held only long enough to read or update a transient state in memory—such as updating a buffer descriptor, traversing a B-Tree index page, or appending to the WAL insert queue.

The array of LWLocks is created dynamically based on system requirements and the number of predicted concurrent processes[17]. The CreateLWLocks() routine establishes an array of lock structures in the shared memory[17]. A critical hardware optimization present in the source code is the padding of these lock structures. The macro LWLOCK_PADDED_SIZE ensures that each lock structure is sized to either 16 or 32 bytes, depending on the architecture[17]. This padding specifically aligns the locks to the size of CPU cache lines, fundamentally preventing "false sharing"—a severe performance degradation phenomenon where multiple CPUs repeatedly invalidate each other's L1/L2 caches because they are modifying independent locks that happen to reside on the same physical memory cache line[17].

The state of all system LWLocks is held centrally. The LWLockAcquire() and LWLockRelease() functions manage access, utilizing an underlying spinlock mechanism[16]. If an LWLock is highly contended, rather than waste CPU cycles spinning, the backend adds its PGPROC identifier to a wait list embedded within the lock's shared memory structure. It then suspends itself via an operating system semaphore (PGSemaphoreCreate()), waiting to be explicitly awakened by the backend that ultimately releases the lock[15].

Heavyweight Locks and the Elastic Fast-Path Mechanism

Unlike LWLocks, Heavyweight Locks are utilized for logical, long-duration database operations, such as establishing isolation over relations (tables) during schema modifications or tracking in-progress transactions. The overarching state of these locks resides in a massive shared hash table located within the ProcGlobal shared memory area[1]. This lock table maps logical Lock Tags (e.g., a specific Object Identifier or OID of a table combined with the lock mode) to the backend processes currently holding or waiting for that lock.

A significant architectural enhancement materialized in the REL_18_STABLE branch involves the Fast-Path locking mechanism. Historically, PostgreSQL hard-coded a strict limit of 16 fast-path locks per backend process[18]. Fast-path locks are an optimization designed to acquire weak relation locks (such as AccessShareLock during standard SELECT queries) directly within a backend's local PGPROC structure without incurring the overhead of traversing and locking the central shared lock table[18]. However, if a backend accessed a 17th relation, the fast-path arrays overflowed, and performance degraded significantly as it fell back to the central lock manager[18].

In PostgreSQL 18, this hard-coded limitation has been eradicated. The size of the fast-path

arrays is now calculated dynamically during the CalculateFastPathLockShmemSize() startup phase[9]. This shared memory structure now scales elastically, allowing modern, highly complex analytical queries that access hundreds of individual table partitions to maintain fast-path lock performance without inducing central lock manager contention[18].

Process State Monitoring and the PgBackendStatus

PostgreSQL requires a highly accessible, centralized location in shared memory to monitor the execution state of all running processes. This is vital for cross-process query cancellation, deadlock detection algorithms, and database administrator monitoring tools (most notably the pg_stat_activity system view).

The PGPROC array represents the primary execution context of every backend process in the system. During initialization, the Postmaster allocates a massive array of PGPROC structures in shared memory, sizing the array based on the max_connections configuration, combined with allowances for autovacuum workers, parallel query workers, and custom background workers[15]. When a child process is spawned via fork(), it claims a free PGPROC slot to advertise its existence to the rest of the cluster[1].

Bound closely to the PGPROC array is the PgBackendStatus structure, which serves a distinct purpose: publishing dynamic execution metrics and query text to the cluster[20]. This structure holds the currently executing query string (st_activity_raw), the exact wait event the process is currently suspended on, and specific integer progress parameters for long-running utility commands (e.g., tracking the phase and blocks scanned during a VACUUM or CREATE INDEX operation via the st_progress_command and st_progress_param variables)[20].

Because pg_stat_activity is queried frequently by monitoring systems, protecting the PgBackendStatus array with traditional LWLocks would introduce unacceptable system-wide contention. To prevent read/write tearing without incurring lock overhead, the PgBackendStatus implementation deploys an elegant optimistic concurrency control mechanism via a sequence counter, st_changecount[20].

When a backend updates its state (e.g., transitioning from "idle" to "active"), it executes a memory barrier (pg_write_barrier()) to prevent out-of-order execution by the CPU pipeline[20]. It then increments st_changecount to an odd number, writes the new state data into the shared struct, executes a second memory barrier, and increments the counter to an even number[20]. Readers observe the counter, copy the memory into process-local space, and check the counter again[20]. If the counter remains even and unchanged from the initial read, the reader is guaranteed that the copy is consistent and no tearing occurred[20]. If the counter is odd or has changed, the reader discards the local copy and loops to try again[20]. This lock-free shared memory pattern optimizes system throughput, prioritizing rapid state publication over strict serialization[20].

The Asynchronous I/O (AIO) Subsystem: Non-Blocking Paradigms

Perhaps the most monumental shift in PostgreSQL 18's shared memory utilization is the full

integration of a native Asynchronous I/O (AIO) subsystem[21]. Historically, PostgreSQL relied completely on synchronous I/O. When a backend required a data block that was not present in the shared_buffers cache, it issued a blocking read() or pread() system call[22]. This operation stalled the CPU thread entirely, waiting passively until the physical disk or network storage responded[22]. The AIO subsystem fundamentally changes this paradigm by overlapping computation with storage retrieval, allowing the backend to request multiple pages and continue processing other data while the kernel fetches the I/O in the background[21].
The shared memory architecture of the AIO subsystem is heavily dependent on the chosen io_method configuration parameter.

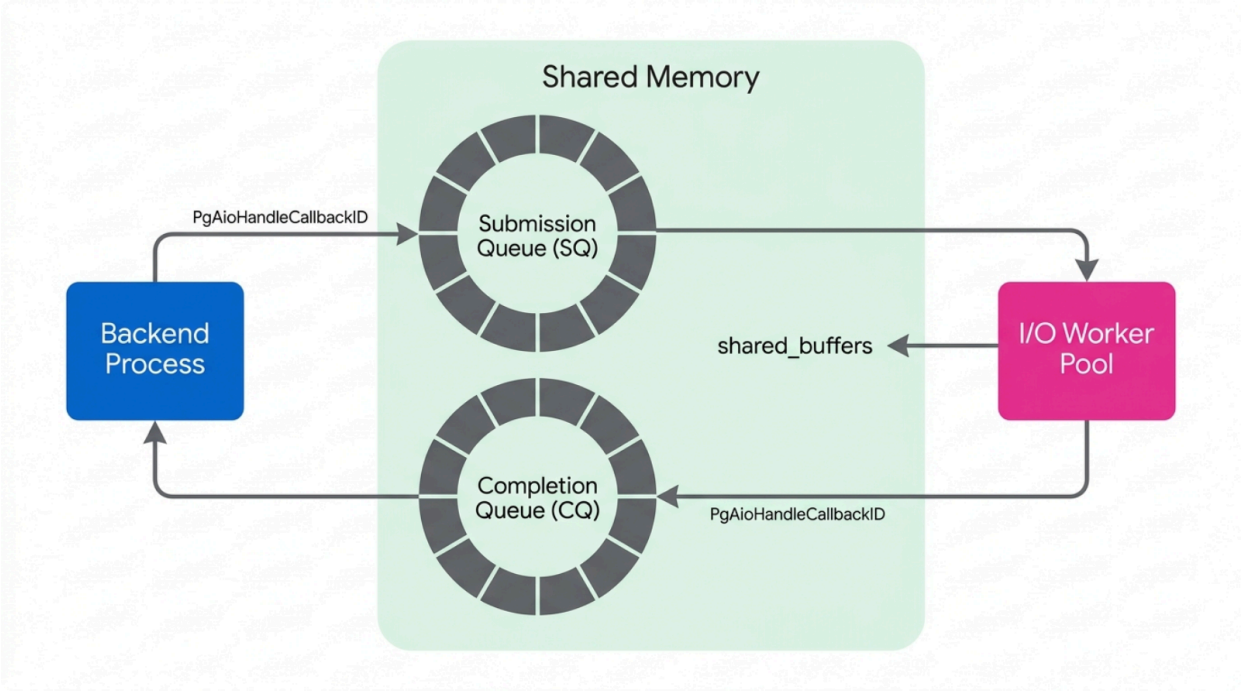
I/O Method Configuration	Architectural Implementation in Shared Memory
sync	The legacy behavior. Serves as a fallback mechanism for compatibility. It utilizes the AIO API pathways but executes synchronously on the backend process. Uses minimal extra shared memory[21].
worker	The portable asynchronous backend. A pool of dedicated background I/O worker processes performs blocking reads on behalf of query backends. The submitter and the worker communicate rapidly over shared memory ring buffers[21].
io_uring	The native Linux backend. Requires no extra PostgreSQL background processes. Utilizes the kernel's native io_uring interface, establishing a shared memory ring directly between the PostgreSQL backend process and the Linux kernel itself[23].

The Ring Buffer Architecture

In the highly portable worker mode, the AIO mechanism constructs its entire queuing infrastructure strictly within PostgreSQL's shared memory[23]. The architecture is designed around non-blocking Ring Buffers—specifically, a Submission Queue (SQ) and a Completion Queue (CQ)[24].
When a backend anticipates needing several data blocks (e.g., during a sequential heap scan, a bitmap heap scan, or a massive VACUUM operation), it prepares read requests and enqueues them into the Submission Queue located in shared memory[21]. These queue entries do not

contain the raw data payload. Instead, they contain only the metadata required for the worker to execute the operation: the RelFileLocator (which file to read), the physical byte offset, the length, and the precise target slot pre-allocated within the shared_buffers pool where the data must be deposited[24].

PostgreSQL 18 AIO Subsystem: Shared Memory Ring Architecture



In 'worker' mode, backends push I/O requests to the shared Submission Queue. Dedicated I/O workers pop these requests, perform the physical disk reads, and push the results to the Completion Queue, waking the backend only when data is ready in the buffer pool.

Callbacks and Completion Processing Constraints

A critical challenge in shared memory design is that C-language function pointers cannot be safely stored within the shared segment. Because different backend processes may map the shared memory at different base virtual addresses (due to ASLR or varying dynamic library loads), a memory address pointing to a function in Backend A may point to unmapped memory or malicious code in Backend B. Therefore, an AIO completion callback cannot just point to a raw memory address in the execution text segment[25].

To resolve this limitation securely, the AIO structures utilize a strict enumeration known as PgAioHandleCallbackID[25]. When an I/O worker completes a physical disk read and writes the data into the buffer pool, it pushes a completion event to the Completion Queue[21]. The

backend process polling the CQ reads this one-byte PgAioHandleCallbackID (e.g., the enum value PGAIO_HCB_SHARED_BUFFER_READV) and routes the completion event through a locally compiled switch statement to the correct handler function[25]. This design allows any backend, or even the I/O worker itself, to safely process completion callbacks within critical sections without risking segmentation faults from invalid pointer dereferencing[26].

This architecture allows the SQL planner to execute massive parallel sequential scans, driven by the io_combine_limit and io_workers GUC parameters[21]. Multiple requested 8KB pages are mathematically combined into single requests inside the shared memory structure, effectively narrowing the massive latency gap between sequential and random I/O costs, and maximizing network-attached storage bandwidth in highly distributed cloud environments[21].

The Cumulative Statistics System (PgStat): The dshash Revolution

The tracking of database statistics—such as the number of tuples inserted, cache blocks hit, or dead rows accumulated—has undergone massive structural changes over recent PostgreSQL releases, culminating in a highly sophisticated shared memory model in version 18. Historically, statistics were collected locally by backends and sent via unreliable UDP packets to a centralized statistics collector process, which periodically serialized them to disk. This proved to be a severe bottleneck on highly concurrent systems, leading to dropped packets and stale monitoring data.

In the modern architecture, the distinct statistics collector process has been entirely eliminated. All operational statistics are now managed directly within the shared memory segment[27].

Fixed vs. Variable Statistics

The system intelligently divides metrics into two operational categories, reflecting two radically different shared memory allocation strategies:

1. **Fixed-numbered stats**: These are singular, global system counters—such as the background writer, checkpointer, and WAL generation statistics. Because there is only ever exactly one instance of these, they are statically allocated in plain, fixed shared memory during the ipci.c initialization phase[27].
2. **Variable-numbered stats**: These track statistics for dynamic, user-created objects like databases, tables, indexes, and logical replication slots[27]. Because an administrator can CREATE and DROP relations arbitrarily, allocating a fixed array for them during startup is impossible.

Dynamic Shared Hash (dshash) and Memory Efficiency

To manage these variable-numbered statistics, PostgreSQL utilizes a dshash (Dynamic Shared Hash) table nested within a Dynamic Shared Area (DSA)[27]. While standard shared memory is permanently fixed at startup, the DSA module allows PostgreSQL to request additional shared memory blocks from the operating system dynamically during runtime to scale with catalog growth[27].

When a backend modifies a table (e.g., an INSERT), it does not immediately write the changes to the dshash in shared memory, as doing so would cause extreme lock contention across thousands of connections. Instead, the backend aggregates the changes in a highly optimized process-local hashtable called pgStatEntryRefHash, allocated within the pgStatPendingContext memory context[27]. This local memory acts as a high-speed buffer containing PgStat_EntryRef structures[27]. Just before transaction commit, or during an explicit idle timeout, the backend flushes these pending statistics en masse to the shared dshash[27].

Each statistical entry in shared memory, defined by structures like PgStat_TableStatus (which tracks elements like delta_live_tuples and delta_dead_tuples), is protected by a highly granular, dedicated LWLock to ensure write consistency[27]. A crucial architectural optimization is that the statistical counters themselves are not stored directly within the PgStatShared_HashEntry header block. Instead, they are separately allocated in the header's body payload. This separation of indexing from data allows the exact same dshash mechanism to handle vastly different payload sizes for different statistic types (e.g., function execution stats versus table tuple stats) without wasting massive amounts of byte padding[27]. This architecture practically eliminates the IPC contention that plagued earlier PostgreSQL versions, allowing cumulative statistics to scale linearly with modern multi-core processor counts[27].

Write-Ahead Logging (WAL) and Replication Coordination

Shared memory serves as the absolute nexus of durability and high availability in PostgreSQL. The Write-Ahead Log (WAL) ensures atomic, consistent, isolated, and durable (ACID) properties, while logical and physical replication mechanisms ensure fault tolerance across geographically dispersed clusters.

While explicitly excluded from the scope of the main shared_buffers data cache, WAL records rely on their own dedicated, highly optimized buffer ring within shared memory: wal_buffers[30]. Before any modification is physically applied to a data page, the exact binary change description is constructed in a backend's local memory and then copied rapidly into the wal_buffers[30]. This shared ring serves as the high-speed staging area where the background WalWriter process sweeps the sequential records and issues the synchronous flush to disk. Space within this specific buffer is highly contended and is protected by WALInsertLock arrays, which allow multiple backends to copy their payload into different slots of the shared wal_buffers simultaneously without blocking each other.

In a high-availability cluster, standby replica nodes run a continuous WalReceiver background process. This process connects to the primary node over TCP/IP to stream WAL records[31]. As the receiver pulls data over the network and flushes it to the local disk, it continuously updates a critical shared memory structure known as WalRcv[31].

The most critical field within this structure is the WalRcv->flushedUpto variable[31]. The local Startup Process—which is responsible for replaying the WAL modifications against the local data files to bring the standby database up to date—constantly monitors this shared variable[31]. The WalRcv structure essentially acts as the primary communication bridge between network

ingestion and local transaction replay[31]. The Startup Process uses this variable to determine precisely how far it can safely apply the log before it must suspend execution and wait for the WalReceiver to fetch more data from the primary[31].

Autovacuum Orchestration

To maintain performance and prevent transaction ID wraparound, PostgreSQL requires regular vacuuming to freeze old transaction IDs and reclaim space from dead tuples. The autovacuum daemon is not a single process; it consists of a central Launcher and multiple Worker processes, which coordinate their autonomous efforts entirely through an Autovacuum shared memory segment[19].

The launcher acts as the master scheduler but does not execute the vacuuming logic itself. Instead, it examines catalog tables, calculates vacuum priorities, and stores its directives into the shared memory area[19]. When a worker process boots, it connects to the shared memory to read its assignment, understanding precisely which database (wi_dboid) and which specific table (wi_tableoid) it has been tasked to process[19].

This shared tracking is crucial to prevent redundant and destructive work overlap. Multiple workers can operate within the same database concurrently, but before a worker attempts to acquire a heavy relation lock, it publishes its current target to the shared memory wi_tableoid array[19]. Other workers check this array and deliberately bypass any table that is currently being processed by their peers[19]. Furthermore, the shared memory facilitates the cost-based vacuum delay feature. The launcher dynamically adjusts the I/O cost limits, balancing the load across all active workers by writing the updated limits to the shared segment. This ensures the combined vacuuming effort does not saturate the underlying storage subsystem, preventing routine maintenance from impacting foreground client queries[19].

Simple LRU (SLRU) Caches

PostgreSQL utilizes several specialized caching mechanisms known as Simple LRU (SLRU) caches. Unlike the generic shared_buffers which cache large 8KB pages of raw user relation data, SLRU caches manage highly dense, specialized metadata critical for transaction visibility and concurrency control[16].

These SLRU caches are allocated during the CreateSharedMemoryAndSemaphores phase and mapped to individual backend processes via the shmem_slru_attach() function[16]. Notable SLRUs integral to the database engine include:

SLRU Cache Name	Engineering Purpose and Shared Memory Function
Commit Log (clog/xact)	A highly dense bit array storing the final commit status (In Progress, Committed, Aborted, Sub-Committed) of every

	Transaction ID (XID)[32]. When a backend checks the visibility of a tuple during an MVCC scan, it queries the CLOG SLRU to verify if the creating transaction successfully committed.
MultiXact	Utilized when multiple independent transactions hold shared row-level locks on the exact same tuple. The MultiXact SLRU stores arrays of XIDs mapped to a single MultiXact ID, preventing severe tuple header bloat on heavily contended rows.
Subtrans	A mapping of nested sub-transaction IDs to their top-level parent transactions. This is absolutely vital for savepoint visibility resolution and complex error handling[29].

Because SLRU cache hits are absolutely required for almost every visibility check in the Multi-Version Concurrency Control (MVCC) model, these shared memory caches must remain highly available. They are protected by specifically tuned ControlLock LWLocks to meticulously manage the eviction and page-in of the SLRU blocks from the underlying disk subsystem[17].

Architectural Diversity in PostgreSQL Shared Memory Allocation

Subsystem	Primary Object	Allocation Strategy	Locking Paradigm	Volatility
PgStat (Statistics)	`PgStatShared_HashEntry` & Fixed Stats	Dynamic Shared Memory (variable) & Plain Shared Memory (fixed)	Dedicated `LWLock` per entry	Flushed periodically or after commit
Asynchronous I/O (AIO)	Submission Queue (SQ) & Completion Queue (CQ)	Shared ring buffers	Lock-free queue / API Queue	Transient (per I/O request)
Lock Manager	Fast-path lock arrays	Dynamically sized at startup (formerly hard-coded)	Shared lock table / Fast-path protocol	High (updated per transaction/lock)
Process Monitoring	`PgBackendStatus`	Fixed arrays in shared memory	Lockless memory barriers (`st_changecount`)	Very High (updated throughout execution)
Autovacuum	Worker information structures	Fixed autovacuum shared memory area	Shared memory flags / Relation Lock	Low to Medium (updated on worker launch/completion)

Comparison of various shared memory subsystems, demonstrating how PostgreSQL tailors allocation methods—from fixed SysV blocks to Dynamic Shared Areas (DSA)—to match the specific scaling demands of each feature.

Data sources: Doxygen, PostgreSQL Mailing List, InterDB, GitHub (pgstat.c), GitHub (autovacuum.c), GitHub (backend_status.h)

Dynamic Shared Memory (DSM) and Parallel Execution

The traditional shared memory allocated during the ipci.c startup sequence is entirely fixed in

size. However, the introduction of Parallel Queries fundamentally required memory that could be allocated, utilized, and completely destroyed on the fly based on dynamic query demands. To resolve this, PostgreSQL implements the Dynamic Shared Memory (DSM) facility[10].
During query execution, the SQL optimizer may decide to split a complex sort, hash join, or massive aggregation across multiple parallel background workers[33]. The Gather node, acting as the leader process, requests a new DSM segment directly from the operating system, totally independent of the primary shared memory block[33].
Within this dynamic segment, PostgreSQL establishes Shared Memory Message Queues (shm_mq)[10]. These lock-free queues form the communication backbone of parallel execution. Worker processes scan their partitioned portions of the data and push tuples into the shm_mq. The leader process continuously polls these shared queues, merging the distinct data streams to return a unified result set to the client[33]. This direct row allocation over shared memory prevents workers from spilling intermediate processing phases to physical disk unnecessarily, efficiently utilizing the available RAM allocated dynamically to the DSM[33]. Upon completion of the query, the entire DSM segment is cleanly unmapped and the memory returned to the operating system, standing in stark contrast to the permanent, immutable structures established during the initial cluster boot.

The Extension Ecosystem: shmem_request_hook

PostgreSQL's renowned extensibility extends deeply into its shared memory architecture. External modules require their own shared memory to function—for example, the ubiquitous pg_stat_statements module requires vast amounts of shared hash tables to aggregate query telemetry (execution times, block hits, rows returned) across all isolated backends.
Historically, extensions aggressively requested and allocated memory during their _PG_init routines when the shared library was loaded. However, modern releases enforce stricter, more deterministic safety paradigms. Extensions must now register their exact memory requirements utilizing the shmem_request_hook[11]. This ensures their needs are mathematically factored into the CalculateShmemSize phase before any OS memory is mapped[4]. Following allocation, extensions use the shmem_startup_hook (shmem_startup_hook_type) to initialize their specific internal hash tables and protective LWLocks using the centralized ShmemInitStruct API[4].
This hook-based architecture guarantees that external binaries, regardless of their complexity or origin, conform absolutely to the precise multi-phase initialization rules dictated by the Postmaster. This ensures that the total shared memory footprint remains strictly immutable once the system transitions to the SRS_DONE state, maintaining the absolute integrity and stability of the entire database cluster under production workloads[11].

Works cited

1. src/backend/postmaster/postmaster.c Source File - PostgreSQL Source Code, https://doxygen.postgresql.org/postmaster_8c_source.html
2. PostgreSQL Internals Deep Dive: From Startup to Crash Recovery, Physical

Replication, https://live.pgconf.in/api/uploads/presentations/67-presentation.pdf
3. Postgres-XC Concept, Implementation and Achievements, https://postgres-x2.github.io/presentation_docs/2014-07-PGXC-Implementation/pgxc.pdf
4. src/backend/storage/ipc/ipci.c File ... - PostgreSQL Source Code, https://doxygen.postgresql.org/ipci_8c.html
5. src/backend/tcop/postgres.c File Reference - PostgreSQL Source Code, https://doxygen.postgresql.org/postgres_8c.html
6. src/include/storage/pg_shmem.h File Reference - PostgreSQL Source Code, https://doxygen.postgresql.org/pg__shmem_8h.html
7. New Module Initialization vs Bootstrap in PostgreSQL - Highgo Software Inc., https://www.highgo.ca/2023/08/11/new-module-initialization-vs-bootstrap-in-postgresql/
8. Is it possible to know the memory being allocated by the method "CreateSharedMemoryAndSemaphores"? - Stack Overflow, https://stackoverflow.com/questions/39607940/is-it-possible-to-know-the-memory-being-allocated-by-the-method-createsharedmem
9. Globals - PostgreSQL Source Code, https://doxygen.postgresql.org/globals_func_c.html
10. src/include/storage/ipc.h File Reference - PostgreSQL Source Code, https://doxygen.postgresql.org/ipc_8h.html
11. Fini for _PG_fini(): recent changes in shared library coding - sql-info.de, https://sql-info.de/postgresql/notes/pg-fini-recent-changes-in-shared-library-coding.html
12. notes/NOTES/greenplum.md at master · xujiabin02/notes - GitHub, https://github.com/xujiabin02/notes/blob/master/NOTES/greenplum.md
13. src/backend/port/sysv_shmem.c File Reference - PostgreSQL Source Code, https://doxygen.postgresql.org/sysv__shmem_8c.html
14. Segment will not start with error "Failed system call was semget" in VMware Tanzu Greenplum - Broadcom support portal, https://knowledge.broadcom.com/external/article/296323/segment-will-not-start-with-error-failed.html
15. Greenplum fails to start with error "could not create semaphores: No space left on device", https://knowledge.broadcom.com/external/article/297014/greenplum-fails-to-start-with-error-coul.html
16. src/backend/storage/ipc/shmem.c File Reference - PostgreSQL Source Code, https://doxygen.postgresql.org/shmem_8c.html
17. PostgreSQL Source Code: src/backend/storage/lmgr/lwlock.c File Reference - Huihoo, https://docs.huihoo.com/doxygen/postgresql/lwlock_8c.html
18. pgsql: Increase the number of fast-path lock slots, https://www.postgresql.org/message-id/E1ss4gX-000IvX-63%40gemulon.postgresql.org
19. postgres/src/backend/postmaster/autovacuum.c at master - GitHub, https://github.com/postgres/postgres/blob/master/src/backend/postmaster/autov

acuum.c
20. postgres/src/include/utils/backend_status.h at master - GitHub, https://github.com/postgres/postgres/blob/master/src/include/utils/backend_status.h
21. PostgreSQL 18 Asynchronous Disk I/O - Deep Dive Into Implementation - credativ GmbH, https://www.credativ.de/en/blog/postgresql-en/postgresql-18-asynchronous-disk-i-o-deep-dive-into-implementation/
22. Supercharging PostgreSQL 18: A Deep Dive into Asynchronous I/O | by Shardul Borhade, https://medium.com/@borhadeshardul/supercharging-postgresql-18-a-deep-dive-into-asynchronous-i-o-3c7bf806f286
23. AIO Grows Up - The Build, https://thebuild.com/blog/aio-grows-up/
24. 8.5. Asynchronous I/O in PostgreSQL - Hironobu SUZUKI @ InterDB, https://www.interdb.jp/pg/pgsql08/05.html
25. src include storage aio.h - PostgreSQL Source Code, https://doxygen.postgresql.org/aio_8h_source.html
26. Asynchronous & Direct IO - PostgreSQL Source Code, https://doxygen.postgresql.org/md_src_2backend_2storage_2aio_2README.html
27. postgres/src/backend/utils/activity/pgstat.c at master - GitHub, https://github.com/postgres/postgres/blob/master/src/backend/utils/activity/pgstat.c
28. Re: shared-memory based stats collector - Mailing list pgsql-hackers - Postgres Professional, https://postgrespro.com/list/id/20190704.192754.27063464.horikyota.ntt@gmail.com
29. postgres/src/include/pgstat.h at master - GitHub, https://github.com/postgres/postgres/blob/master/src/include/pgstat.h
30. Обсуждение: Re: How about a psql backslash command to show GUCs? - Postgres Pro, https://postgrespro.ru/list/thread-id/2597145
31. postgres/src/backend/replication/walreceiver.c at master - GitHub, https://github.com/postgres/postgres/blob/master/src/backend/replication/walreceiver.c
32. Обсуждение: Global snapshots : Компания Postgres Professional, https://postgrespro.ru/list/thread-id/2385436
33. How does postgres allocate rows to workers and use memory in parallel queries? [closed], https://stackoverflow.com/questions/78668620/how-does-postgres-allocate-rows-to-workers-and-use-memory-in-parallel-queries
34. blog-3/201708/20170807_02.md at master - GitHub, https://github.com/debug001/blog-3/blob/master/201708/20170807_02.md